III. Bölüm - Öz Değer ve Öz Vektör ve PCA

Öz değer ile Öz Vektör Nedir?

* Bir matris, vektörler üzerinde dönme, uzama, kısalma ve yansıma gibi dönüşümler gerçekleştirebilir.
* Ancak bazı vektörler, matrislerle çarpıldığında aynı doğrultuda kalır. Bunlara **öz vektör** ne kadar ölçeklendirileceğine **öz değer** denir.
* Temel formül:

$$Av = \lambda . v$$

- Burda
  - A = Dönüşüm matrisi
  - v = Özvektör
  - λ = Öz değer

Örnek: $A = \begin{bmatrix} 3 & 0 \\ 0 & 1 \end{bmatrix}$ $v_1 = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$ $v_2 = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$ $v_3 = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$

Verilen vektörlerin öz değerlerini bulunuz.

1. için

$$\begin{bmatrix} 3 & 0 \\ 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 3 \\ 0 \end{bmatrix} \qquad \begin{bmatrix} 3 \\ 0 \end{bmatrix} = \begin{bmatrix} 1 \\ 0 \end{bmatrix} \cdot \lambda_1$$

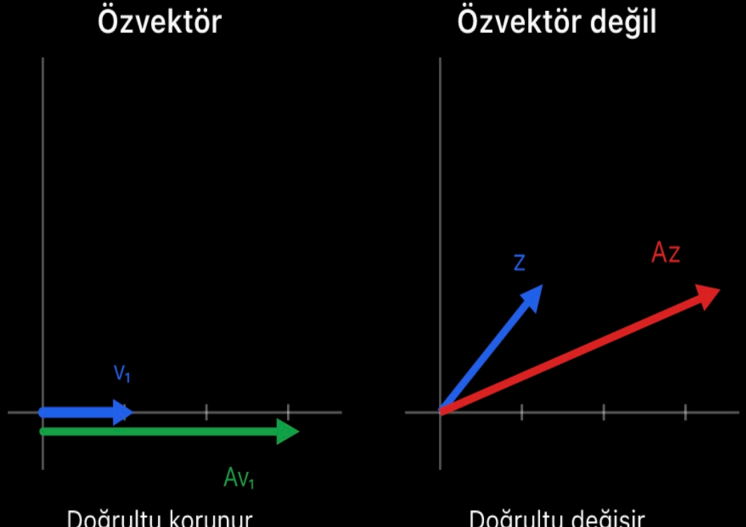
$$\lambda_1 = 3$$

2. için: $\begin{bmatrix} 3 & 0 \\ 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} 0 \\ 1 \end{bmatrix} = \begin{bmatrix} 0 \\ 3 \end{bmatrix}$ $\lambda_1 = 3$

$\begin{bmatrix} 3 & 0 \\ 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} 1 \\ 1 \end{bmatrix} = \begin{bmatrix} 3 \\ 1 \end{bmatrix} = \lambda_1 \begin{bmatrix} 1 \\ 1 \end{bmatrix}$, 3 vektör öz vektör değildir.

* Öz değerin geometrik anlamı:

Öz değer	Etki
$\lambda > 1$	Vektör uzar
$\lambda = 1$	Değişmez
$0 < \lambda < 1$	Kısalır
$\lambda = 0$	Sıfır vektöre dönüşür
$\lambda < 0$	Doğrultu korunur ters döner.



Şekil 1 — Ders materyalindeki 4. sayfa örneğinin şematik gösterimi.

Kovaryans Matrisi ve Öz Vektör İlişkisi

* Kovaryans, iki değişkenin birbirini nasıl değiştirdiğini gösterir.
* Örneğin; çalışma süresi ile sınav arasında başarı genellikle birlikte artabilir. Böylece veride pozitif kovaryans görülebilir.
* İki değişkenli kovaryans matrisi:

$$C = \begin{bmatrix} var(x) & cov(x,y) \\ cov(y,x) & var(y) \end{bmatrix}$$

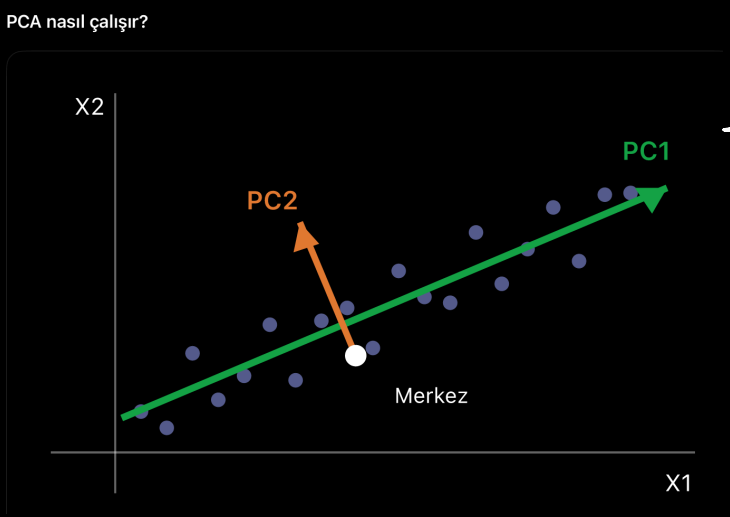
* Merkezleşmiş veri matrisi X_E için:

$$C = \frac{1}{n-1} X_c^T \cdot X_c$$

- Köşegen elemanları varyanstır.
- Köşegen dışındakiler kovaryanstır.
- Matris simetriktir.
- Öz vektörleri verinin değişim yayılımını gösterir.
- Öz değerleri bu yönlerde varyansı hesaplanır.

PCA - Temel Bileşen Analizi

* PCA (Principal Component Analysis), çok boyutlu bir veri kümesinin mümkün olduğunca fazla varyansını koruyarak daha az boyuta indiren doğrusal yöntemdir.
* Örneğin; 100 farklı özelliği olan veri kümesinde, bu özellikleri yüksek ilişki varsa daha az sayıda bileşenle ifade eder.



* Burda:
  - PC1 (Birincil Temel Bileşen): Verinin en büyük varyansını açıklayan
  - PC2 (İkincil "). PC1'e dik olan ve geriye en yüksek varyansı taşıyan vektör.

$$P_{C1} \cdot PC2 = 0$$

* PCA, mevcut özellikleri doğrudan silmez. Bunun yerine doğrusal birleşimden yeni özellik oluşturur.

PCA Temel Adımları

1. **Centering**
   $X_c = X - \mu$
2. **Covariance**
   Kovaryans matrisi
3. **Eigen decomposition**
   Özdeğer ve özvektör
4. **Sıralama**
   Büyük özdeğer önce
5. **Bileşen seçimi**
   İlk k yön
6. **Projection**
   Yeni boyuta izdüşüm

1. Centering - Merkezleme

- Her özelliden kendi ortasını çıkarır

$$x_c = x - \mu$$

Örnek: $x = \begin{bmatrix} 2 & 4 \\ 4 & 6 \\ 6 & 8 \end{bmatrix}$, $\mu = [4, 6]$

$$x_c = \begin{bmatrix} -2 & -2 \\ 0 & 0 \\ 2 & 2 \end{bmatrix}$$

2. Kovaryon hesaplama

- $C = \frac{1}{n-1} x_c^T \cdot x_c$

3. Öz değer ve öz vektör bulma

$$A \cdot v = \lambda \cdot v$$

4. Öz değerlerini sıralama

- En büyük öz değer, en fazla varyans taşıyan yönü temsil eder.

5. İlk k bileşeni seçme

- İki boyutu tek boyuta indirme

6. Projection - İz Düşüm

Numpy ile sıfırdan PCA hesaplama

$$X = \begin{bmatrix} 2 & 1 \\ 3 & 2 \\ 4 & 2 \\ 5 & 4 \\ 6 & 5 \end{bmatrix}$$

```python
import numpy as np

# 1. Veri matrisi
X = np.array([
    [2, 1],
    [3, 2],
    [4, 2],
    [5, 4],
    [6, 5]
], dtype=float)

# 2. Ortalama ve merkezleme
mean = X.mean(axis=0)
Xc = X - mean

# 3. Kovaryans matrisi
C = np.cov(Xc, rowvar=False)

# 4. Özdeğer ve özvektörler
values, vectors = np.linalg.eigh(C)

# 5. Büyükten küçüğe sırala
idx = np.argsort(values)[::-1]
values = values[idx]
vectors = vectors[:, idx]

# 6. İlk bileşeni seç
k = 1
Vk = vectors[:, :k]

# 7. Yeni eksene izdüşüm
Z = Xc @ Vk

# 8. Açıklanan varyans
ratio = values / values.sum()

print("Ortalama:", mean)
print("Kovaryans:\n", C)
print("Özdeğerler:", np.round(values, 4))
print("Özvektörler:\n", np.round(vectors, 4))
print("Varyans oranları:", np.round(ratio, 4))
print("Yeni veri:\n", np.round(Z, 4))
```

Büyüklük	Değer
Birinci Öz Değer	5.102
İkinci " "	0,098
Pc1 açıklanan Varyans	%98.12
Pc2 " "	%1,88
Önceki boyut	5x2
Sonraki boyut	5x1

Explaid Variance (Açıklanan Varyans)

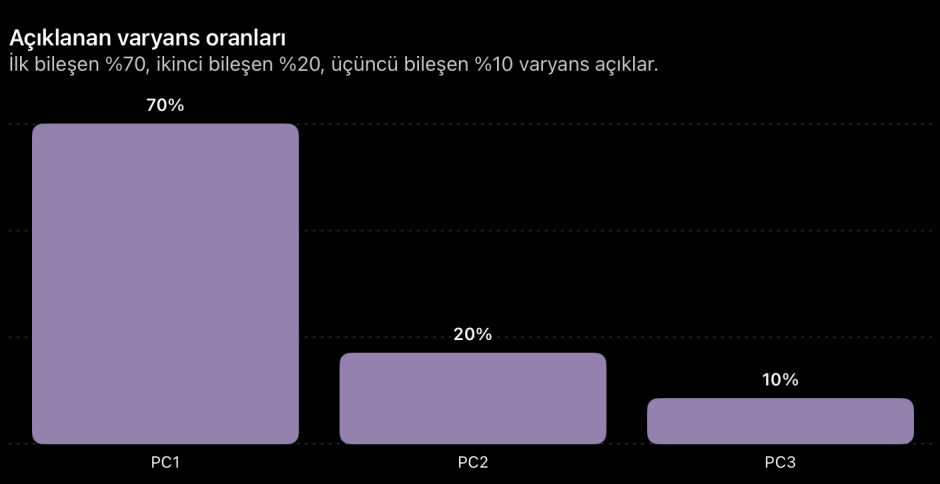
* Bir pcA bileşenleri toplamı ne kadar açıklandığını ifade eder

$$EVR_i = \frac{\lambda_i}{\sum_j \lambda_j}$$

Örnek: $\lambda_1 = 7 \quad \lambda_2 = 2 \quad \lambda_3 = 3$

Toplam varyon = 10

$$EVR_1 = \frac{7}{10} \qquad EVR_2 = \frac{2}{10} \qquad EVR_3 = \frac{1}{10}$$



* Kumulatif açılan varyon ile ilk k bileşeni açıklandığı oran

PC1 ⟶ 70

PC2+PC3 → 30

P1 + PC2 + PC3 → 100

Scikit-learn ile PCA

```python
from sklearn.decomposition import PCA
import numpy as np

X = np.array([
    [2, 1],
    [3, 2],
    [4, 2],
    [5, 4],
    [6, 5]
], dtype=float)

pca = PCA(n_components=1)
Z = pca.fit_transform(X)

print("PC1:", pca.components_)
print("Özdeğer:", pca.explained_variance_)
print("Açıklanan varyans:",
      pca.explained_variance_ratio_)
print("Yeni veri:\n", np.round(Z, 4))

# Veriyi yaklaşık olarak geri kurma
X_hat = pca.inverse_transform(Z)
print("Geri kurulmuş veri:\n", X_hat)
```

*Burda:
- np.linalg.eigh() sonunda öz vektör sutunlarını bulur.
- Sklearn icindeki pca.components_ ise satırlarını bulur.
- pca.explaind_varince = öz değerleri verir.
- pca.explaind_varince_ratio oranları verir.

*Reconstruction - Veriyi geri kurtarma
- Boyut indirgemeden önce verilere yaklaşık dönüşüm yapılır.

$$\hat{X} = Z V_k^T + \mu$$

8. Feature Scaling ve PCA

PCA, değişkenlerin ölçeğine duyarlıdır.

Örneğin:

- Yaş: 18–60
- Gelir: 15.000–100.000 TL

Gelirin sayısal büyüklüğü çok daha fazla olduğu için PCA'nın ilk bileşeni gelir yönüne ağırlık verebilir.

Bu sorunu çözmek için standardizasyon kullanılabilir.

$$z = \frac{x - \mu}{\sigma}$$

Standardizasyon sonrasında özelliklerin ortalaması sıfır, standart sapması bir olur (eğitim verisinde kullanılan tanıma göre).

Python

```python
import numpy as np
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA

X = np.array([
    [20, 15000],
    [25, 25000],
    [30, 40000],
    [40, 60000],
    [50, 90000]
], dtype=float)

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

pca = PCA(n_components=1)
Z = pca.fit_transform(X_scaled)

print("Standardize veri:\n",
      np.round(X_scaled, 3))

print("PC1:", pca.components_)
print("Açıklanan varyans:",
      pca.explained_variance_ratio_)
```

9. Data Leakage — Veri sızıntısı

PDF'in 19–20. sayfalarındaki önemli uyarı: PCA ve ölçekleyici, makine öğrenmesi uygulamalarında yalnızca eğitim verisi üzerinde `fit` edilmelidir.

Hatalı akış

Tüm veri → Scaler + PCA fit → Train/Test

Doğru akış

Veri → Train/Test ayrımı

Train: fit_transform | Test: transform

Pratikte veri sızıntısını önlemek için `Pipeline` kullanılabilir:

Python

```python
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split

# X: özellik matrisi, y: sınıf etiketleri
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42,
    stratify=y
)

model = Pipeline([
    ("scaler", StandardScaler()),
    ("pca", PCA(n_components=0.95)),
    ("model", LogisticRegression(max_iter=1000))
])

model.fit(X_train, y_train)
print("Test başarısı:", model.score(X_test, y_test))
```

Bu kod, X ve y değişkenlerinin bulunduğu bir sınıflandırma veri kümesiyle çalıştırılmalıdır. `n_components=0.95`, PCA'nın en az %95 varyansı açıklayan sayıda bileşeni seçmesini sağlar.

10. PCA'nın avantajları ve sınırlılıkları

Avantajları	Sınırlılıkları
Boyut sayısını azaltır.	Bilgi kaybı oluşturabilir.
Model eğitimini hızlandırabilir.	Doğrusal yapıların ötesini yakalayamaz.
Görselleştirmeyi kolaylaştırır.	Özellik ölçeklerinden etkilenir.
Tekrarlanan bilgiyi azaltabilir.	Yeni bileşenlerin yorumu zordur.
Gürültüyü azaltabilir.	Aykırı değerlere duyarlıdır.

Önemli: PCA en büyük varyansı seçer ancak yüksek varyans, sınıflandırma açısından en yararlı bilgi anlamına gelmez. Ders PDF'inin 18. sayfasındaki örnekte PC1 varyansın %90'ını taşımasına rağmen sınıfları ayıramaz; PC2 ise kalan %10 ile sınıfları ayırır.